Day 3

101 Sequence detector using Moore FSM

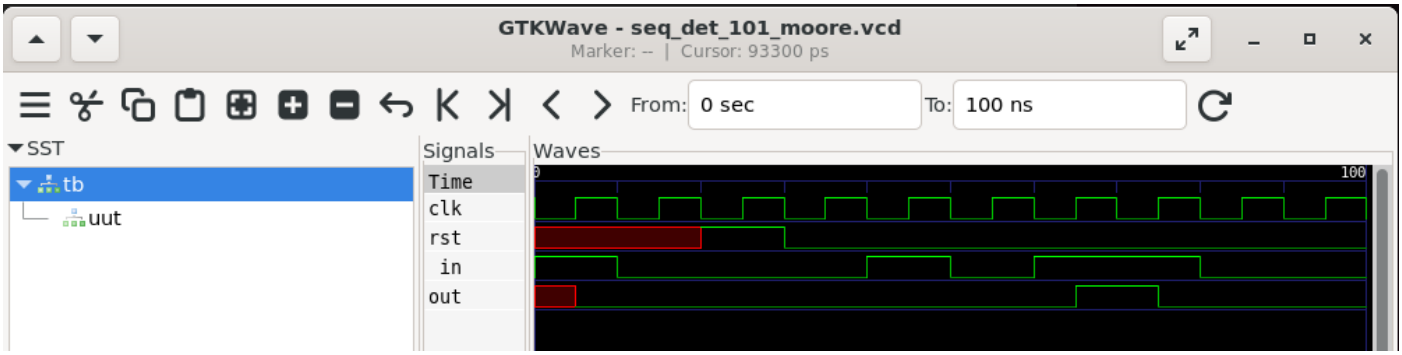
Verilog Code:

```
module seq_det_101_moore(input clk, rst, in,
 output reg out);

parameter IDLE = 2'b00,
 S1 = 2'b01,
 S10 = 2'b10,
 S101 = 2'b11;

reg[1:0] state, n_state;

always @(posedge clk or posedge rst) begin
 if(rst)
  state <= IDLE;
 else
 state <= n_state;
end

always @(*) begin
case(state)
 IDLE : n_state = in? S1 : IDLE;
 S1 : n_state = in? S1 : S10;
 S10: n_state = in? S101 : IDLE;
 S101: n_state = in? S1 : S10;
 default: n_state = IDLE;
endcase
end

always @(*) begin
 out = (state == S101) ? 1 : 0;
end
endmodule
```

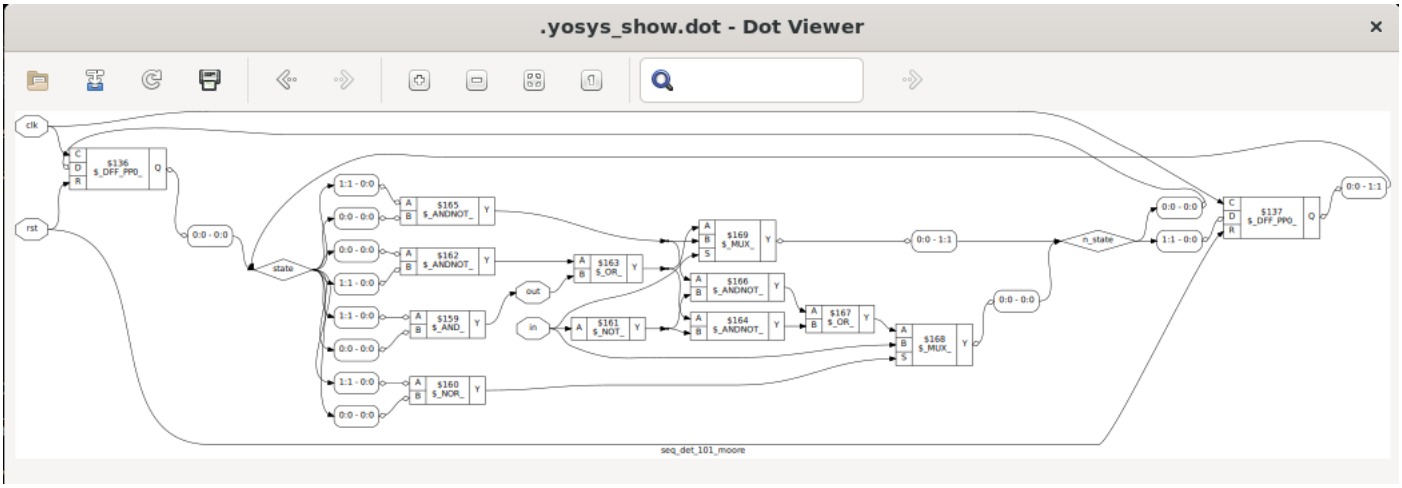
Testbench:

```
`timescale 1ns/1ps
module tb;
reg clk, rst, in;
wire out;

seq_det_101_moore uut(.clk(clk), .rst(rst), .in(in), .out(out));

initial begin
 $dumpfile("seq_det_101_moore.vcd");
 $dumpvars(0, tb);
end

initial begin
 clk = 0;
 forever #5 clk = ~clk;
end

initial begin
 in = 1; #10;
```

```
 in = 0; #10;
 rst = 1; #10;
 rst = 0; #10;
 in = 1; #10;
 in = 0; #10;
 in = 1; #10;
 in = 1; #10;
 in = 0; #10;
 in = 0; #10;
$finish;
end
endmodule
```



Simulation and Synthesis output:

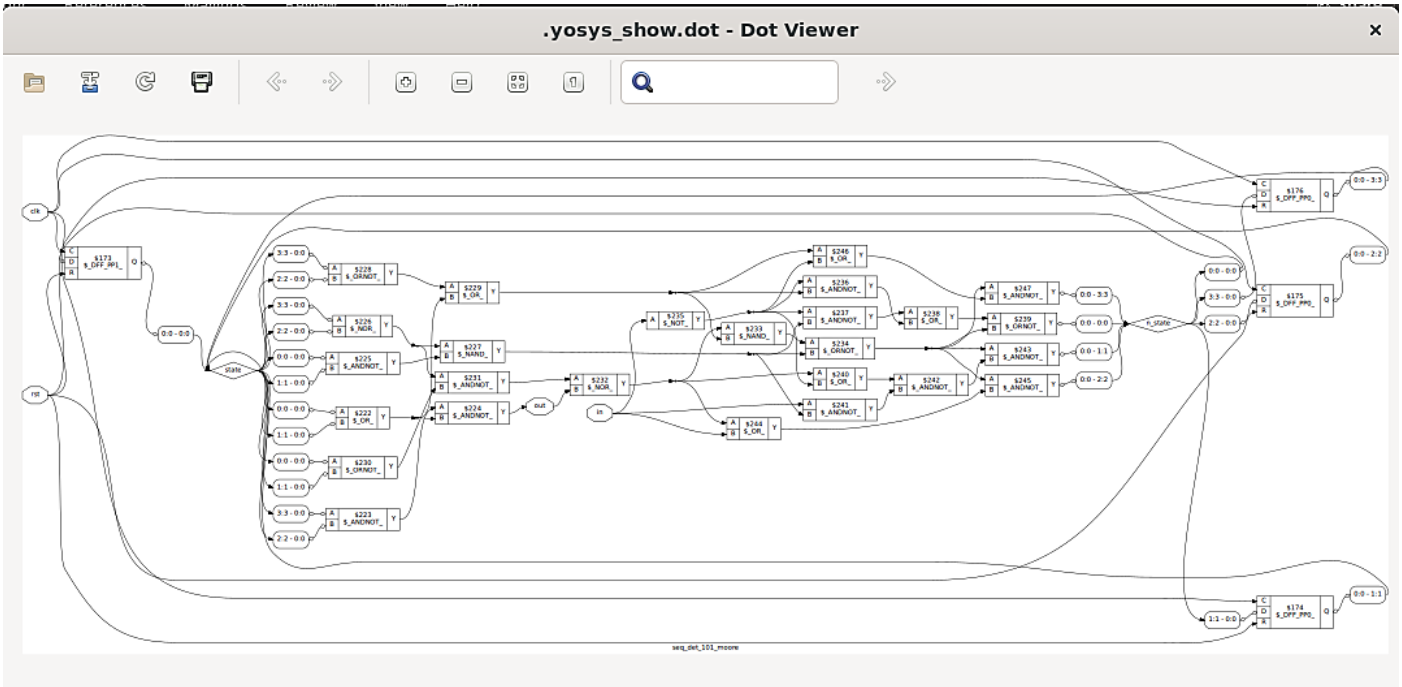


2.26. Printing statistics.

```
=== seq_det_101_moore ===

   Number of wires:                 14
   Number of wire bits:             16
   Number of public wires:           6
   Number of public wire bits:       8
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                 13
     $_ANDNOT_                       4
     $_AND_                          1
     $_DFF_PP0_                      2
     $_MUX_                          2
     $_NOR_                          1
```

$_NOT_ 1
$_OR_ 2

Same code using One hot encoding method



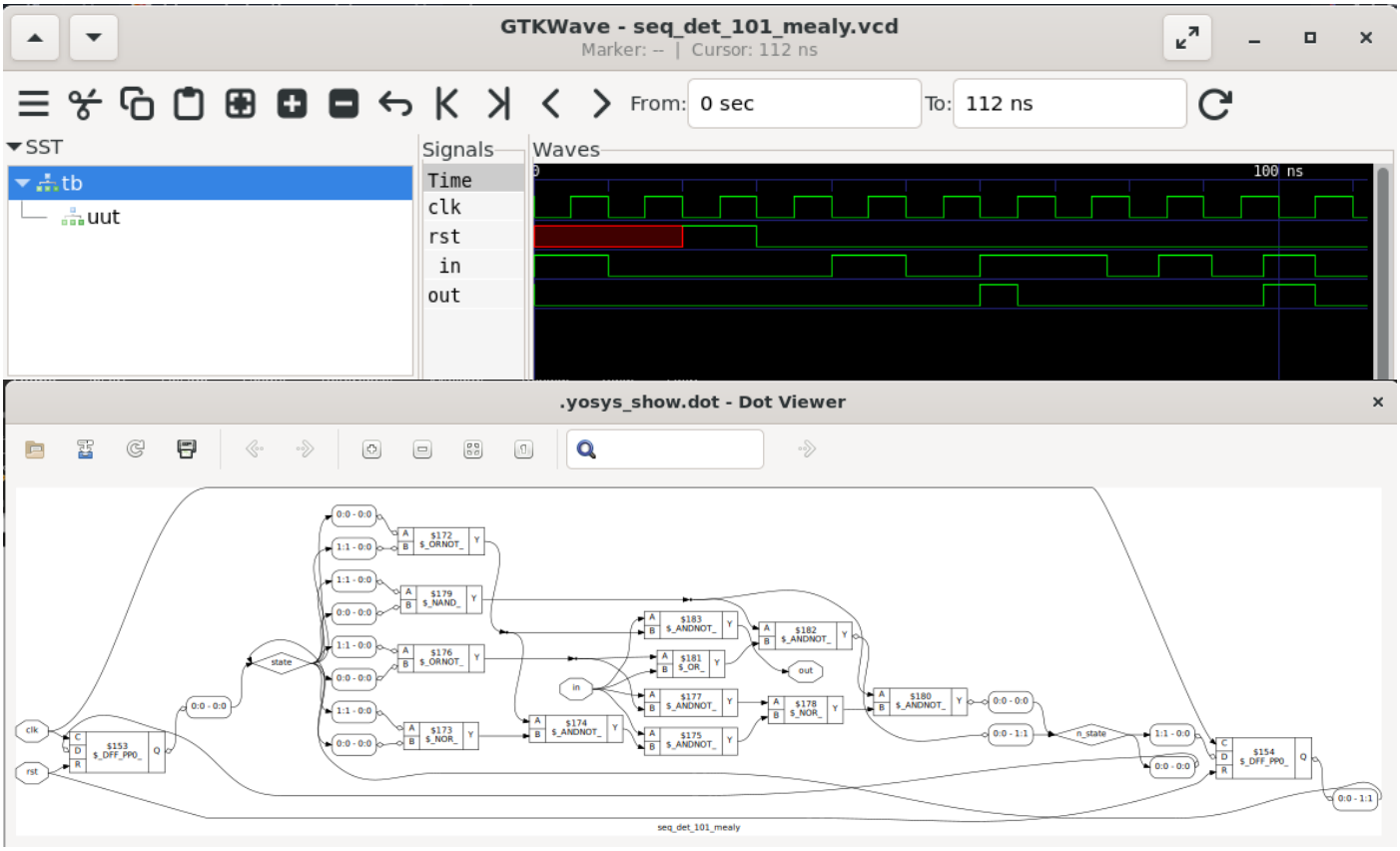
2.26. Printing statistics.

=== seq_det_101_moore ===

Number of wires: 27
Number of wire bits: 33
Number of public wires: 6
Number of public wire bits: 12
Number of memories: 0
Number of memory bits: 0
Number of processes: 0
Number of cells: 30
$_ANDNOT_ 11
$_DFF_PP0_ 3
$_DFF_PP1_ 1
$_NAND_ 2
$_NOR_ 2
$_NOT_ 1
$_ORNOT_ 4
$_OR_ 6

101 Sequence Detector using Mealy FSM:

```
`timescale 1ns/1ps
module seq_det_101_mealy (
   input clk, rst, in,
   output reg  out
);
   parameter IDLE = 2'b00,
            S1   = 2'b01,
            S10  = 2'b10;

   reg [1:0] state, n_state;

   // State register
   always @(posedge clk or posedge rst) begin
      if (rst)
         state <= IDLE;
      else
         state <= n_state;
   end
   always @(*) begin
      n_state = state;
      out = 0;

      case (state)
         IDLE: begin
            if (in)
               n_state = S1;
            else
               n_state = IDLE;
         end

         S1: begin
            if (in)
               n_state = S1;
            else
               n_state = S10;
         end

         S10: begin
            if (in) begin
               n_state = S1;
               out = 1;       // Mealy: output immediately
            end else
               n_state = IDLE;
         end

         default: n_state = IDLE;
      endcase
   end

endmodule
```



2.26. Printing statistics.

=== seq_det_101_mealy ===

Number of wires: 15
Number of wire bits: 17
Number of public wires: 6
Number of public wire bits: 8
Number of memories: 0
Number of memory bits: 0
Number of processes: 0
Number of cells: 14
$_ANDNOT_ 6
$_DFF_PP0_ 2
$_NAND_ 1
$_NOR_ 2
$_ORNOT_ 2
$_OR_ 1

Important Insights:

In the first half, I learnt that using a smaller number of code/ digits to represent states requires very less hardware, as seen between binary coding and one hot coding techniques.

Using Mealy and Moore type FSMs, Mealy fsm changes the output when there is a change in state along with an immediate change in the current input, while Moore fsm only depends on the states rather than inputs